Integration Flows Design

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the technical design of each integration flow between Kommo CRM and Aloware.

Unlike the Architecture Design document, which describes the solution at a high level, this document focuses on how individual synchronization processes are structured and how they interact with the participating systems.

The objective is to establish a consistent design for every integration flow before implementation.

Scope

This document describes:

- Integration flows
- Trigger mechanisms
- Participating systems
- Data direction
- High-level processing sequence
- Expected outputs
- Design considerations

It intentionally excludes implementation details such as API endpoints, field mappings, validation rules, retries, logging, and Make module configuration.

Flow Design Principles

Every integration flow should follow the same architectural principles.

Independent Execution

Each flow operates independently from all others.

A failure in one flow must not interrupt unrelated synchronizations.

Event-Driven Processing

Flows are initiated by business events whenever webhook support is available.

Polling should only be used when no event mechanism exists.

Stateless Processing

Every execution should contain all information required to complete the synchronization.

No workflow should depend on execution state stored within Make.

Deterministic Processing

Given the same input, a flow should always produce the same expected result.

Idempotent Operations

Whenever possible, flows should safely support repeated execution without creating duplicate business effects.

Standard Flow Lifecycle

Every integration flow follows the same logical sequence.

```
Business Event
       |
       ▼
  Trigger Reception
       |
       ▼
  Input Validation
       |
       ▼
  Data Retrieval
       |
       ▼
  Data Transformation
       |
       ▼
  Target System Update
       |
       ▼
  Result Validation
       |
       ▼
  Logging
       |
       ▼
  Flow Completion
```

This lifecycle is shared by every synchronization process.

Integration Flow Catalog

Flow 01 — Kommo → Aloware

Objective

Synchronize commercial events from Kommo into Aloware so communication campaigns always operate with current CRM information.

Source System

Kommo CRM

Target System

Aloware

Trigger

Business events generated in Kommo.

Examples include:

- Lead stage changes
- Lead creation
- Lead updates
- Contact updates
- Owner assignment

High-Level Process

1. Receive business event.
2. Validate payload.
3. Retrieve additional CRM information if required.
4. Transform business data.
5. Send data to Aloware.
6. Validate response.
7. Record execution outcome.

Expected Result

Aloware reflects the latest commercial information required for outbound communication.

Flow 02 — Aloware → Kommo

Objective

Synchronize communication events generated in Aloware back into Kommo.

Source System

Aloware

Target System

Kommo CRM

Trigger

Communication events generated in Aloware.

Examples include:

- Call completed
- SMS sent
- SMS received
- AI Summary generated
- Call recording available

High-Level Process

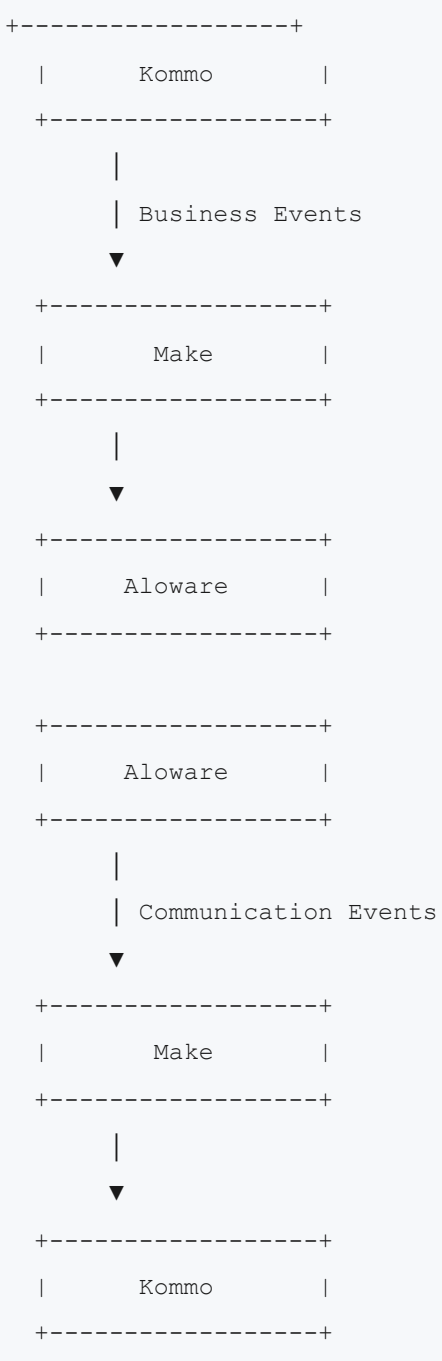
1. Receive communication event.
2. Validate payload.
3. Retrieve additional communication information if necessary.
4. Transform communication data.
5. Update Kommo.
6. Validate update.
7. Record execution outcome.

Expected Result

Kommo contains an accurate communication history associated with each lead.

Flow Interaction Model

Although both synchronization directions are independent, they form a complementary integration.



The two synchronization directions remain logically separated while sharing the same orchestration platform.

Common Processing Stages

Every flow consists of the following conceptual stages.

Stage	Purpose
Trigger Reception	Receive business event
Validation	Verify event integrity
Data Retrieval	Obtain required information
Transformation	Convert source data into target format
Synchronization	Execute target operation
Verification	Confirm successful execution
Logging	Register execution details
Completion	Finish workflow

Flow Boundaries

Each flow owns its complete execution lifecycle.

Responsibilities include:

- Receiving events
- Processing data
- Updating the target system
- Recording operational information

Responsibilities explicitly excluded from individual flows include:

- Global monitoring
- Retry policies
- Security enforcement
- Authentication management
- Cross-flow coordination

These concerns are addressed by dedicated design documents.

Scalability

The architecture allows additional flows to be incorporated without modifying existing ones.

Examples include:

- Contact synchronization
- Company synchronization

- Appointment synchronization
- AI event synchronization
- Marketing automation
- Reporting integrations

Each new flow should follow the same lifecycle and design principles defined in this document.

Design Assumptions

The flow design assumes:

- Reliable webhook delivery when supported.
- REST APIs as the communication mechanism.
- Platform-specific authentication.
- Independent execution of every synchronization.
- Eventual consistency between systems.

Related Documents

This document should be read together with:

- Architecture Design
- Synchronization Model
- Field Mapping
- Stage Mapping
- Agent Mapping
- Data Transformation Rules
- Validation Rules
- Error Handling Strategy
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Security Considerations